



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS

Graduação em Engenharia de Software

Izabela Cecilia Silva Barbosa

780998

Trabalho de Bad Smell

Disciplina: Teste de Software

Belo Horizonte

2026

1. Análise de Smells

A análise do código ReportGenerator.js e de sua suíte de testes revelou falhas estruturais e de design que comprometem a qualidade do software. Abaixo, destaco os três principais problemas encontrados:

1. Alta Complexidade Cognitiva (Long Method / God Object)

- Local: Método generateReport.
- Problema: O método possuía um nível de complexidade cognitiva de 27 (o limite recomendado é 15). Ele acumulava responsabilidades demais: formatar CSV, formatar HTML, aplicar regras de negócio (filtros e prioridades) e calcular totais.
- Risco: Dificuldade imensa de leitura, manutenção e alta probabilidade de introduzir bugs ao modificar o código.

2. Estruturas Condicionais Aninhadas (Collapsible Ifs)

- Local: Método generateReport (linhas 43-45).
- Problema: Uso de blocos `if` alinhados diretamente dentro de outros blocos `if` sem necessidade. A lógica de filtro de usuários e a lógica de geração de formato se misturavam de forma profunda.
- Risco: Aumenta a complexidade ciclomática e prejudica a clareza do fluxo de execução.

3. Teste Frágil (Fragile Test)

- Local: Teste "deve gerar um relatório HTML completo para Admin".
- Problema: O teste dependia de uma falha de formatação do código original. O código gerava uma tag `|` com um espaço extra (`<tr >`) quando o item não possuía o atributo style. O teste exigia esse espaço exato em vez de validar a estrutura semântica ou o comportamento.
| |
- Risco: Qualquer limpeza ou refatoração inofensiva no código de produção quebra a suíte de testes, gerando falsos negativos e perda de tempo.

2. Processo de Refatoração

A refatoração focou na redução de complexidade do código de produção e na correção da fragilidade identificada nos testes.

Refatoração do Código de Produção (Extract Method)

O método gigantesco `generateReport` foi dividido em métodos menores com responsabilidades únicas: `\_generateHeader`, `\_filterItems`, `\_generateRow` e `\_generateFooter`. Isso zerou a ofensa de Complexidade Cognitiva apontada pelo SonarJS/ESLint.

```
// Trecho do código refatorado extraído (Exemplo limpo)
_generateRow(reportType, user, item) {
  if (reportType === 'CSV') {
    return `${item.id},${item.name},${item.value},${user.name}\n`;
  }
  if (reportType === 'HTML') {
    // Correção: o espaço só é adicionado se o style existir.
    const styleAttr = item.priority ? ' style="font-weight:bold;": ";
    return
    `<tr${styleAttr}><td>${item.id}</td><td>${item.name}</td><td>${item.value}</td></tr>\n`;
  }
}
```

```
return ";\n}\n
```

Correção do Teste Frágil

Ao corrigir a formatação das tags no código de produção (removendo o espaço extra do ``<tr >``), os testes quebraram. A decisão correta foi refatorar a asserção no arquivo de teste para não depender desse vício do código legado.

```
// ANTES: Teste dependente de formatação errada\nexpect(report).toContain('<tr ><td>1</td><td>Produto A</td><td>300</td></tr>');\n\n// DEPOIS: Teste consertado e robusto\nexpect(report).toContain('<tr><td>1</td><td>Produto A</td><td>300</td></tr>');
```

3. Relatório da Ferramenta

As falhas arquiteturais foram detectadas de forma automatizada pelo ESLint/SonarJS.

Resultado da Primeira Execução (Problemas Encontrados)

```
/workspaces/bad-smells-js-refactoring/src/ReportGenerator.js\n11:3 error Refactor this function to reduce its Cognitive Complexity from 27 to the 15 allowed\nsonarjs/cognitive-complexity\n43:14 error Merge this if statement with the nested one\nsonarjs/no-collapsible-if\n\n✖ 2 problems (2 errors, 0 warnings)
```

Resultado da Execução Após Refatoração

```
npx eslint src/ReportGenerator.refactored.js\n(retornou vazio, indicando que nenhum bad smell foi encontrado)
```

Comentário sobre a Automação

A ferramenta SonarJS acoplada ao ESLint foi fundamental para apontar pontos cegos de design (como a alta complexidade) que passariam despercebidos numa suíte com 100% de testes passando. Após a divisão das responsabilidades (Extract Method), a ferramenta atestou a limpeza do código.

4. Conclusão

A execução deste trabalho provou que testes rodando com sucesso (`npm test`) não significam que o software tenha qualidade interna. O "Teste Frágil" engessava o código original, pois qualquer correção na formatação do HTML quebrava a integração.

O ciclo de refatorar o código-fonte para reduzir complexidade (apoiado pelo linter) e, simultaneamente, consertar os testes para removerem vícios e fragilidades garante uma manutenção segura. Somente código de produção limpo testado por uma suíte robusta oferece verdadeiro valor e agilidade para uma equipe de engenharia.